

PROJE MİMARİSİ VE TASARIM KARARLARI SAVUNMA RAPORU

Kapsam: Yazılım Mühendisliği İleri Düzey (PhD/MSc) Proje Savunması

Proje: AI Decision Ecosystem Engine (Çoklu Ajanlı Karar Destek Sistemi)

Tarih: 21 Nisan 2026

İÇİNDEKİLER

- Mimari Yaklaşım: Temiz Mimari (Clean Architecture)
- Sistem Dağıtımı: Neden Monolitik (Modular Monolith)?
- Eşzamanlılık Modeli: Asenkron I/O ve Performans
- Yapay Zeka Mimarisi: Nöro-Sembolik (Hibrid) Yaklaşım
- Tasarım Desenleri (Design Patterns) ve SOLID
- Tuzak Jüri Soruları ve Defans Argümanları

1. Mimari Yaklaşım: Temiz Mimari (Clean Architecture)

Sistemimiz Robert C. Martin'in **Clean Architecture** prensiplerine uygun olarak 4 katmanlı (Layered) şekilde tasarlanmıştır. İç içe geçen çemberler modeline uygun olarak, dış katmanlar (Infrastructure, Presentation) iç katmanları (Domain, Application) bilir ancak iç katmanlar dışarıdan tamamen habersizdir (Dependency Rule).

- Presentation Layer (API):** Yalnızca HTTP isteklerini (FastAPI) karşılar, veriyi doğrular (Pydantic) ve Application katmanına iletir.
- Application Layer (Use Cases):** Sistemin iş akışlarını (Senaryo oluşturma, tur tabanlı simülasyonu başlatma) yönetir.
- Domain Layer (Entities & Rules):** Sistemin kalbidir. CEO, CFO, HR ajanlarının matematiksel iş kuralları ve veri modelleri burada yer alır. Veritabanı veya framework kütüphanesi içermez.
- Infrastructure Layer:** Dış dünya ile iletişim kurulan yerdir. Asenkron veritabanı bağlantısı (**asyncpg**), LLM entegrasyonu (Ollama) ve Repository implementasyonları buradadır.

Karar Analizi (Trade-off): Bu mimari başlangıç aşamasında daha fazla dosya ve soyutlama (boilerplate) yazmayı gerektirir. Ancak sağladığı **Test Edilebilirlik (Testability)** çok yüksektir. İş mantığımız veritabanından bağımsız olduğu için 80'den fazla testi bellek (memory) üzerinde 2 saniyenin altında çalıştırabiliyoruz.

2. Sistem Dağıtımı: Neden Monolitik (Modular Monolith)?

Projenin bileşenleri ayrı ayrı microservice'ler olarak değil, **Modular Monolith** (Modüler Tek Parça) mimari üzerinden değerlendirilmiş ve uygulanmıştır.

Neden Microservices (Mikroservis) Kullanılmadı? Yazılım mühendisliğinde her sistem mikroservis olmak zorunda değildir. Hatta domain sınırları (bounded contexts) tam oturmamış sistemlerde mikroservis kullanımı **"Distributed Monolith (Dağıtık Monolitik)"** anti-pattern'ına yol açar.

1. **Ağ Gecikmesi (Network Latency):** Ajanlarımız "Tur Tabanlı (Round-based)" haberleşmektedir. HR ajanının karar verebilmesi için CEO ve CFO'nun verileri anlık gereklidir. Bunu mikroservislere bölmek gereksiz HTTP/gRPC gecikmelerine neden olurdu.
2. **Transaction Yönetimi:** Senaryo analiz sonuçları ve ajan skorları tek bir transaction (işlem) bütünlüğünde kaydedilmelidir (ACID özellikleri). Mikroserviste Saga veya 2PC desenleri kullanmak gereksiz "Over-engineering (Aşırı Mühendislik)" yaratırdı.

Kısacası, kod tabanımız Clean Architecture ile modülerleştirildiği için, gelecekte sadece belli bir modülü ölçeklemek istersek servisleri sorunsuzca dışarı çıkarabiliriz. Şimdilik fiziksel (deployment) olarak monolitik ancak mantıksal (logical) olarak tamamen modülerdir.

3. Eşzamanlılık Modeli: Asenkron I/O ve Performans

Python backend ekosisteminde geleneksel olarak WSGI (Senkron) yapılar kullanılırken, biz tamamen **ASGI (Asenkron - async/await)** bir mimari tasarladık ([asyncpg](#), [FastAPI](#), [SQLAlchemy AsyncSession](#)).

Arkasındaki Mühendislik Fikri: Karar Destek Sistemleri yoğun matematiksel hesaplamalardan ziyade **I/O Bound (Girdi/Çıktı Bağımlı)** operasyonlar barındırır.

- Büyük Dil Modeline (Ollama) ağ üzerinden HTTP isteği atılması ve beklenmesi.
- PostgreSQL veritabanına büyük boyutlu JSON/Metin verilerinin yazılması.

Thread tabanlı senkron bir mimaride (örn. Flask + psycopg2), sistem LLM'den yanıt beklerken işlemci thread'i "Block (Bloke)" olurdu. Asenkron "Event Loop" mimarimiz sayesinde, LLM'in düşünme süresi boyunca sunucu diğer kullanıcıların HTTP isteklerini karşılamaya devam eder, kaynak kullanımı (CPU Utilization) maksimize edilir.

4. Yapay Zeka Mimarisi: Nöro-Sembolik (Hibrid) Yaklaşım

Günümüzde LLM'leri sisteme entegre etmenin en tehlikeli yolu sisteme doğrudan karar yetkisi vermektir. Biz ise yazılım mühendisliği literatüründe **Neuro-Symbolic (Nöral + Sembolik İşleme)** olarak bilinen hibrit (melez) yaklaşımı kullandık.

- **Sembolik (Deterministik) Karar:** CEO, CFO, HR ajanlarımızın vereceği "Onay/Red (Stance)" kararı ve Güven Skoru (Confidence) kesin kurallarla ([if/else](#), matematiksel eşikler) sınırlanmıştır.
- **Nöral (Stokastik) Yorumlama:** Ollama üzerinden koştan Yerel Büyük Dil Modeli (Qwen2.5:14b) **asla karar merci değildir**. Yalnızca deterministik kararın gerekçesini (Reasoning) yönetime açıklayacak doğal dil sentezini gerçekleştirir.

Güvenlik ve Gizlilik Parametresi (Data Privacy): Sektörde kurum içi (On-Premise) kararlar, stratejik şirket sırları barındırır. Model olarak OpenAI GPT-4 kullanmak yerine açık kaynak "Local LLM - Ollama" tercih edilerek verinin kurum dışına sızması (Data Leakage) mimari seviyede engellenmiştir (Privacy-by-Design).

5. Tasarım Desenleri (Design Patterns) ve SOLID

Yazılım geliştirme sürecimizde Gang of Four (GoF) tasarım desenleri (design patterns) ve SOLID adımları yoğun şekilde kullanıldı:

1. **Repository Pattern:** Veritabanına erişim işlemleri soyutlandı. Uygulama hiçbir zaman direkt olarak SQL sorgusu atmaz, **ScenarioRepository** (ve diğer repolar) arayüzleri ile işlem yapar (Dependency Inversion / Loose Coupling).
2. **Factory Pattern (AgentFactory):** İstemci (Client) ajanların nasıl üretildiğini bilmez. **AgentFactory.create_default_agents()** çağrısı ajan nesneleri setini oluşturur. Sisteme yeni ajan eklenmek istendiğinde (örn. Legal Agent) client kodu değiştirilmez.
3. **Singleton Pattern (llm_logger):** Loglama operasyonları sistem geneline yayılmadan tek bir örnekti üzerinden asenkron yönetilmek üzere tasarlandı. Mükemmel bellek yönetimi sağladı.

6. Tuzak Jüri Soruları ve Defans Argümanları

Olası Soru / Eleştiri	Teknik Savunma (Defans)
Soru: Neden ORM kullandınız? Ham SQL (Raw SQL) kullansaydınız daha performanslı olmaz mıydı?	Cevap: ORM kullanımı küçük bir performans maliyeti (overhead) getirse de; uygulama kodumuzu SQL Injection saldırılarından tamamiyle korur, veritabanı objelerinin Python sınıfları olarak tip güvenli (type-safe) kullanılmasını sağlar ve bakım geliştirme eforlarını (maintainability) dramatik bir şekilde düşürür. Performans darboğazları olsaydı ORM seviyesinde indeksleme ve eager loading stratejileri kullanırdık.
Soru: Veritabanında tabloları neden CREATE TABLE ile elle oluşturmadınız da Alembic gibi ekstra bir araç yüklediniz?	Cevap: Gelişmiş veri mimarilerinde şema evrimi (Schema Evolution) izlenebilir olmalıdır. Yazılımda kodun versiyon kontrolü Git ne ise, veritabanı şemasının versiyon kontrolü de Alembic (Database Migrations) aracıdır. Bu, CI/CD süreçlerini otomatize etmemizi sağlar. Takım içi "Benim bilgisayarımdaki DB güncel değil" karmaşasını engeller.
Soru: Ollama (LLM) çökünce veya cevaplayamadığında sistem neden patlamadı?	Cevap: Yazılım mimarisinde Fault Tolerance (Hata Toleransı) gözetilmiştir. LLM'in vereceği "Timeout" veya servisin kapalı olması ihtimalleri Python içinde try-except fallback senaryosuyla sarılmış, exception durumlarında LLM yerine bir Python f-string formatı şablon olarak iade edilmiştir. Sistem "Graceful Degradation (Zarifçe Yıkım)" prensibiyle çökmeden hizmet kalitesini bir tık düşürerek çalışmaya devam eder.

Sonuç: AI Decision Ecosystem, sadece güncel hevelerle kodlanmış basit bir betik dizisi değil; ölçeklenebilirliği, güvenliği ve katmanlı yapısı göz önünde bulundurularak projelendirilmiş akademik olgunlukta bir endüstriyel karar yazılımıdır.